ECSE-6320 Advanced Computer Systems
Project 1 Report

Cooper Werner
Rensselaer Polytechnic Institute
wernec6@rpi.edu
RIN: 662058426

September 16, 2026

Abstract

Contents

1 Introduction **2**

2 Pre-Experiment Hypotheses **2**

3 Correctness Validation **3**
3.1 Validation Method 3
3.2 Validation Results 3

4 Measurement Discipline **3**
4.1 Experimental Setup 3
4.2 Statistical Treatment 4

5 Results **4**
5.1 Summary Figure 4
5.2 Secondary Figures 4
5.3 Results Table 4
5.4 Relevant Code 4

6 Negative / Failure Case **5**

7 Honest Anomalies **5**

8 Discussion **5**

9 Conclusion **5**

A Additional Data / Raw Logs **5**

1 Introduction

The goal of this project is to analyze the performance of SIMD Instructions and how their performance is affected by the datasize, size of the vector, memory alignment, data size, and compiler optimization.

To do this I will run multiple experiments across three different kernels:

1. Vector Multiply and Accumulate: $\vec{y} = a \times \vec{x} + \vec{y}$
2. Vector Elementwise Multiplication: $z_i = x_i * y_i$
3. 3-Element 1D Stencil: $y_i = a \times x_{i-1} + b \times x_i + c \times x_{i+1}$

For each kernel I will adjust the size of the vector using different vectoriziation instructions (SSE, AVX, AVX512). Adjust memory alignment, element size (FP32 vs FP64), number of elements provided, and compiler flags for autovectorization.

Performance will be evaluated using the Linux `timespec` to measure execution time of the vector execution while isolating overhead from I/O operations, and verification.

2 Pre-Experiment Hypotheses

Pre-Experiment Hypothesis

Hypothesis:

The compiler's autovectorization will produce code that runs significantly faster than the serial version. Since the compiler doesn't have the same knowledge as my code as I do, I think my SIMD implementation will be at least the same speed as the autovectorized version or a bit faster.

Reasoning:

The compiler will target the largest vectors that are available on my CPU since it will be provided with the `-march=native` compiler flag. Although my kernels are fairly simple vector operations, since I have better knowledge of my data layout, I think that my handwritten vectoriziation will perform better than the compiler's vectoriziation.

Predicted quantitative outcome:

Table 1: Predicted SIMD speedup relative to scalar baseline

Vectoriziation Strategy	FP32 Speedup from Serial	FP64 Speedup from Serial
Serial	$1x$	$1x$
Compiler Autovectorization	$10.5x$	$5.2x$
Handwritten Vectoriziation	$11x$	$5.6x$

Pre-Experiment Hypothesis

Hypothesis:

Shifting the memory alignment off of the 64 Byte alignment that is preferred by AVX512 will

significantly affect the performance of the SIMD instructions. The instructions will likely still run, however they will not run efficiently and may even be slower than the scalar counterpart.
Reasoning:

CPUs are most efficient when the data is alligned to the memory boundary since CPUs will fetch memory multiple bytes at a time. If the memory is shifted off the memory alignment boundary it will have to do multiple data fetches to fill the buffers. This will increase the number of pipeline stalls and reduce instruction throuhput.
Predicted quantitative outcome:

Table 2: Predicted SIMD Slowdown relative to SIMD with 0 byte offset

Offset Bytes	**FP32 Slowdown**	**FP64 Slowdown**
1	∼0.5×	∼0.25×
2	∼0.5×	∼0.25×
3	∼0.5×	∼0.25×
4	∼1×	∼0.25×
5	∼0.5×	∼0.25×
6	∼0.5×	∼0.25×
7	∼0.5×	∼0.25×

3 Correctness Validation

3.1 Validation Method

3.2 Validation Results

Table 3: Correctness validation summary

Check	Expected	Observed

4 Measurement Discipline

4.1 Experimental Setup

- **Hardware:** CPU: x86_64 AMD Ryzen AI 7 350 w/ Radeon 860M (SIMD Support: SSE, SSE2, AVX, AVX2, AVX-512), Memory: 32GB LPDDR5 @ 5600MT/s
- **OS / Kernel:** Pop_OS! 24.04 Running Linux 7.0.11-76070011-generic
- **Compiler & Flags:** GCC 16.2.0, Flags: -O2 -march=native -ftree-vectorize (disabled for serial experiments)
- **Isolation techniques:** Ran on CPU0 with 100% affinity, and -20 niceness (ensure no context switching)

- **Trials per condition:** 5 trials per condition
- **Timer / Instrumentation:** clock_gettime(CLOCK_MONOTONIC)

4.2 Statistical Treatment

5 Results

5.1 Summary Figure

Figure 1: Caption describing the primary result.

5.2 Secondary Figures

(a) (b)

Figure 2: Side-by-side comparison of . . .

5.3 Results Table

Table 4: Quantitative results

Configuration	Mean	Std. Dev.	N

5.4 Relevant Code

```c
int main(void) {
    // ...
    return 0;
}
```

Listing 1: Benchmark kernel (C)

```cpp
#include <chrono>

int main() {
    auto start = std::chrono::high_resolution_clock::now();
    // ...
}
```

Listing 2: Measurement harness (C++)

```rust
fn main() {
    let start = std::time::Instant::now();
    // ...
}
```

Listing 3: Benchmark harness (Rust)

6 Negative / Failure Case

Negative / Failure Case

What failed:

Evidence:

Root cause (if known):

7 Honest Anomalies

Honest Anomalies

Anomaly 1:

Anomaly 2:

8 Discussion

9 Conclusion

A Additional Data / Raw Logs